# Learning Neural Nets

DSC 340 Applied Machine Learning

April 21, 2023

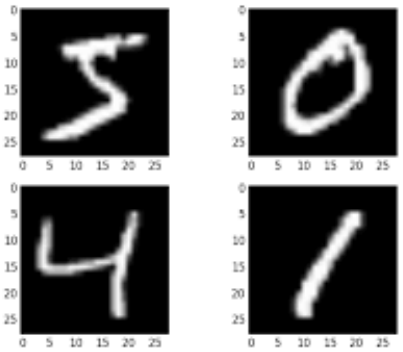
Unit: Deep Learning #2

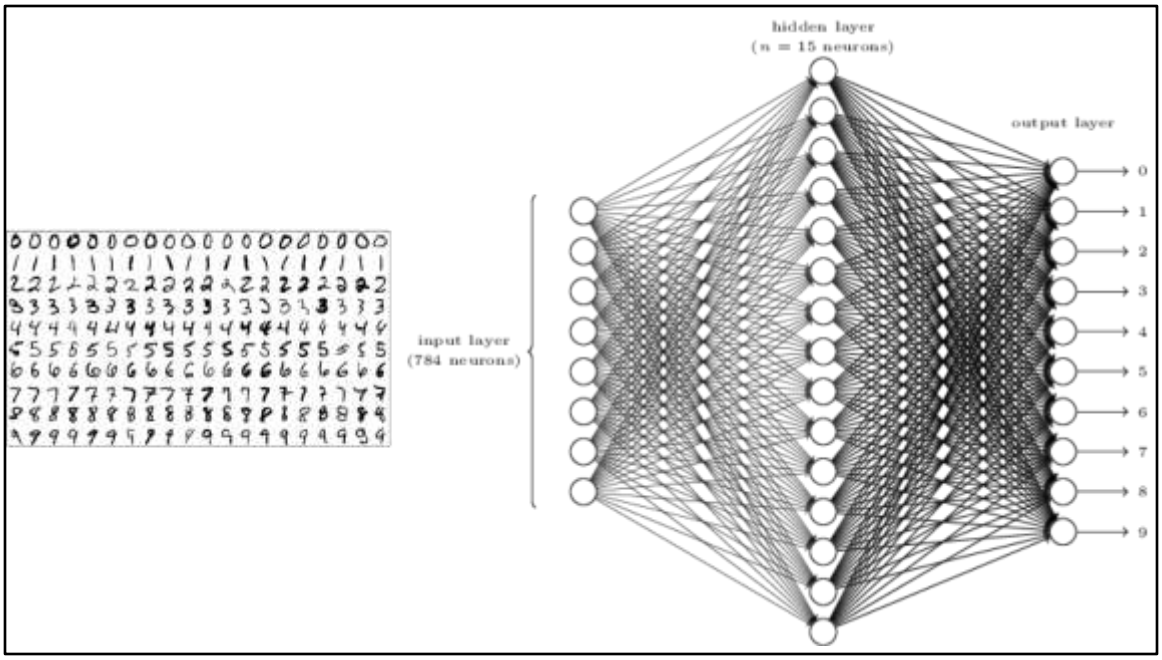
# The Plan

- Continue feedforward neural net
- Method of gradient descent
- Backpropagation in neural nets

# Supervised Learning

- Example: digit recognition
- X instances, features (pixels)
- y class values (e.g., 0, 1, …, 9)
- Classification problem
- Use data to train model (train, test)
- Minimize error (cost or loss)



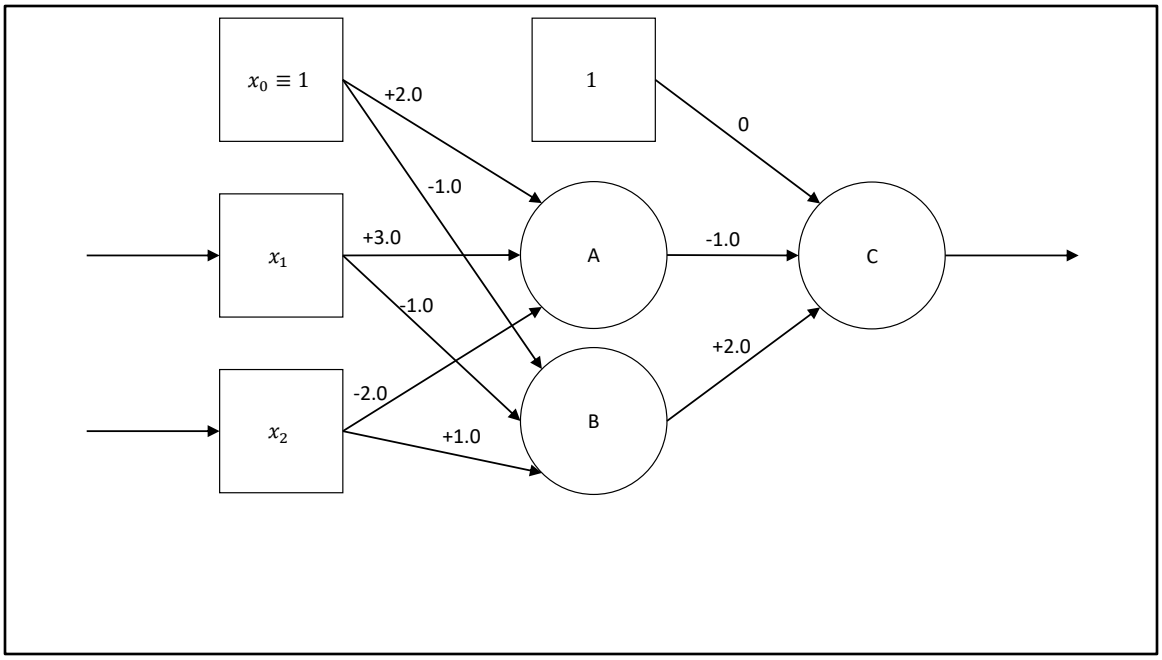

hidden layer
(n = 15 neurons)
output layer
input layer
(784 neurons)
0
1
2
3
4
5
6
7
8
9





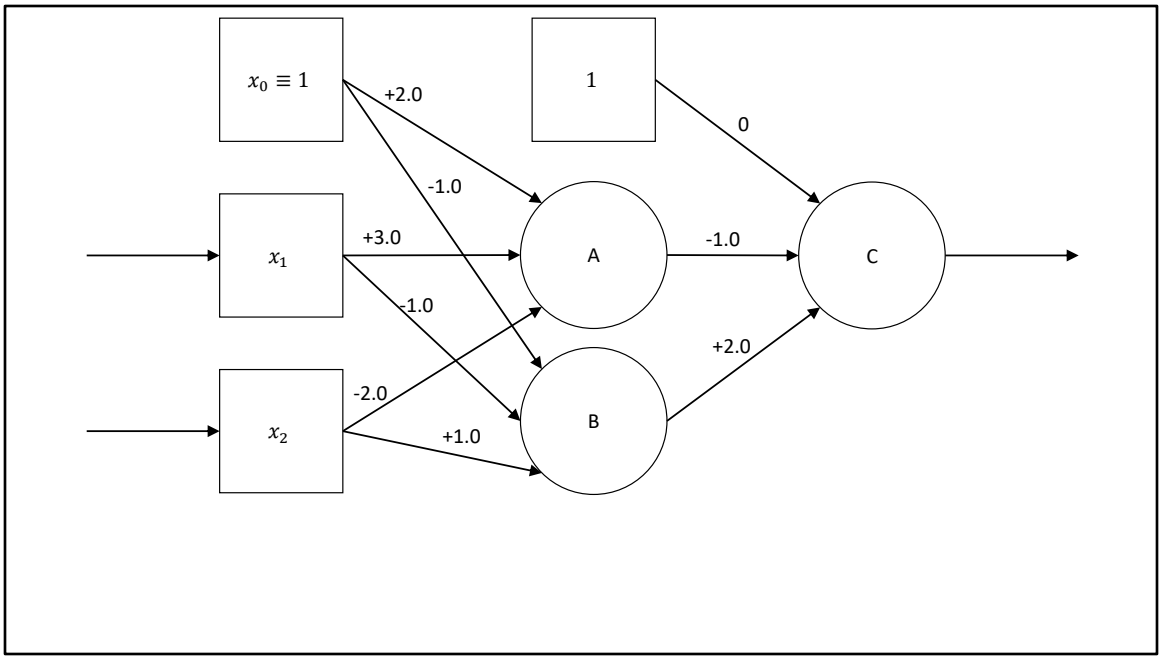
# Neural nets (MLP)

Review from last time


$x_0 \equiv 1$
1
+2.0
0
-1.0
$x_1$
+3.0
A
-1.0
C
-1.0
+2.0
$x_2$
-2.0
B
+1.0

## Terms

- Unit "Neuron" node $z = w_0 + x_1 w_1 + x_2 w_2 = \mathbf{x} \cdot \mathbf{w}$
- Weight "Synapse" edge
- Activation function "fire"


$x_0 \equiv 1$
1
+2.0
0
-1.0
$x_1$
+3.0
A
-1.0
C
-1.0
+2.0
$x_2$
-2.0
B
+1.0

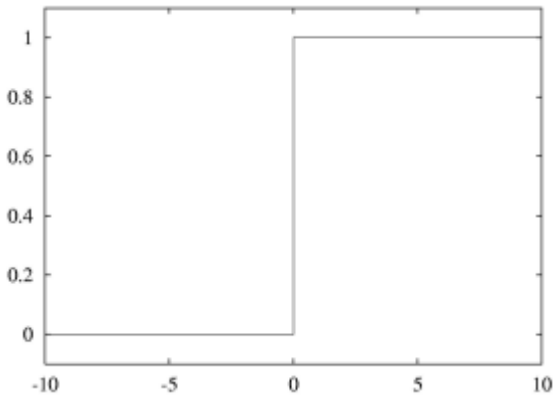
# Terms

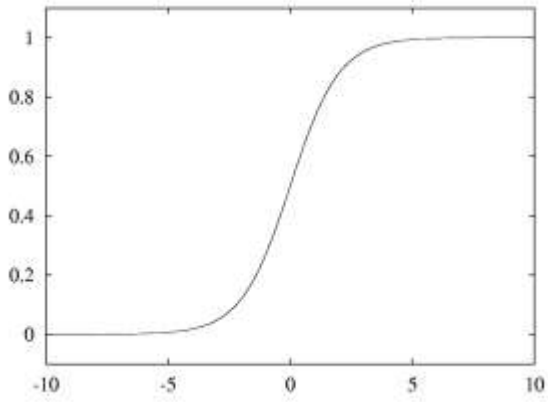
- Neuron (node)
- Synapse (edge)
- Activation function
  - Sigmoid (output: binary)
  - Softmax (output: multiclass)
  - ReLU (hidden layers)

$$z = w_0 + x_1 w_1 + x_2 w_2 = \mathbf{x} \cdot \mathbf{w}$$

$$a = \frac{1}{1 + e^{-z}}$$

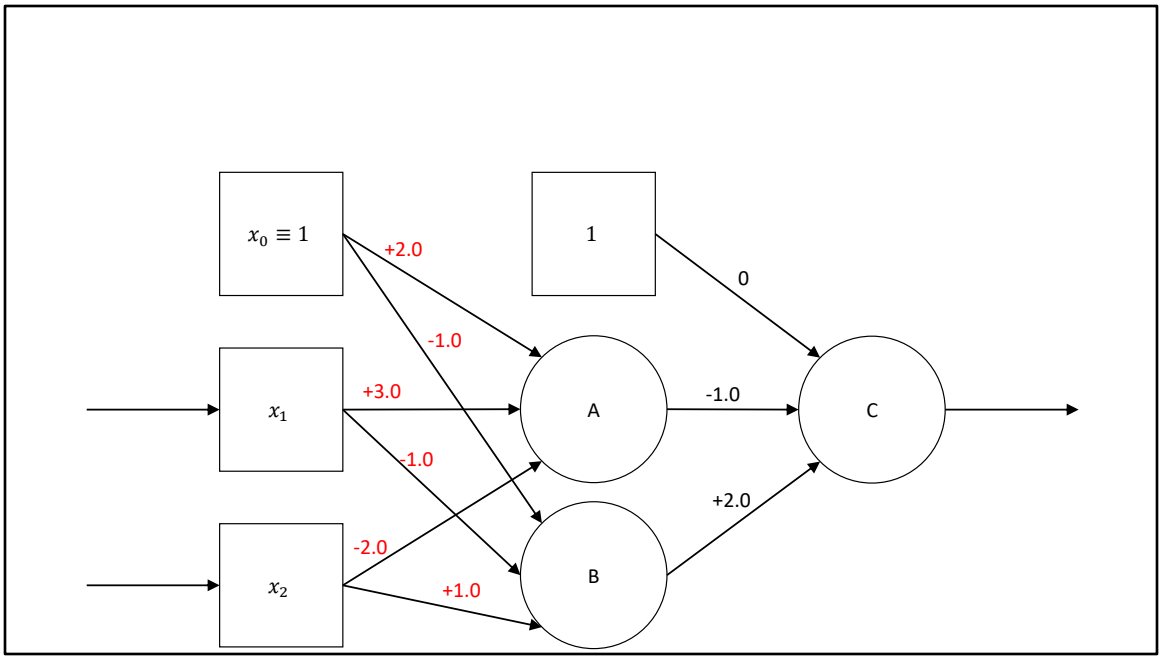
# Threshold vs. sigmoid activation function

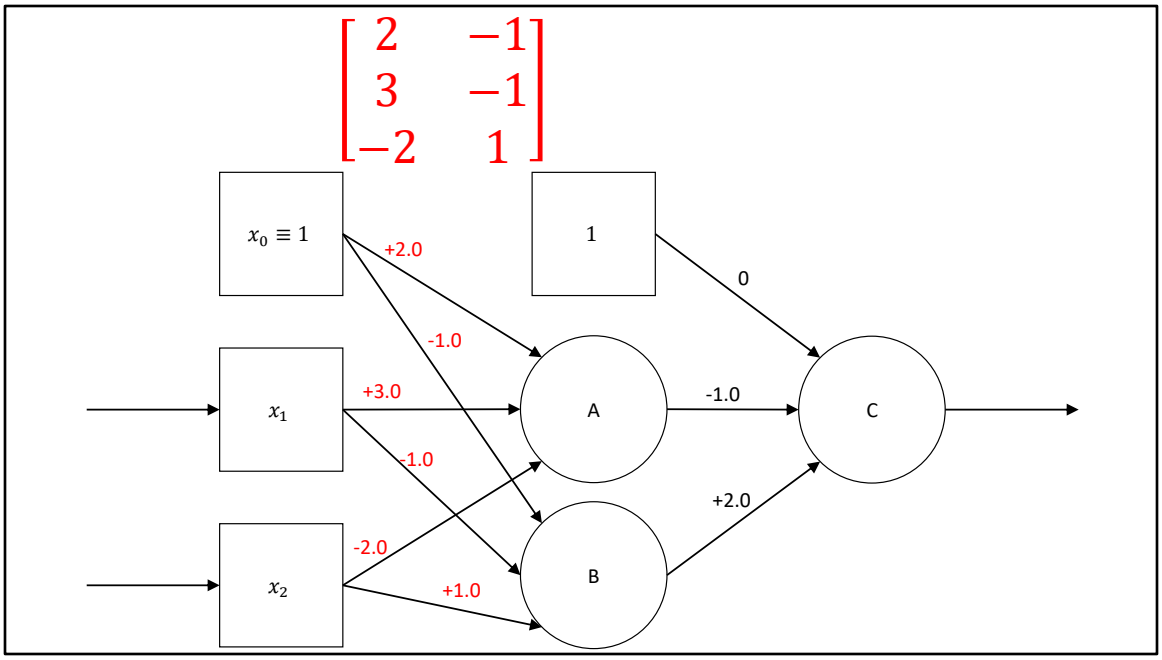


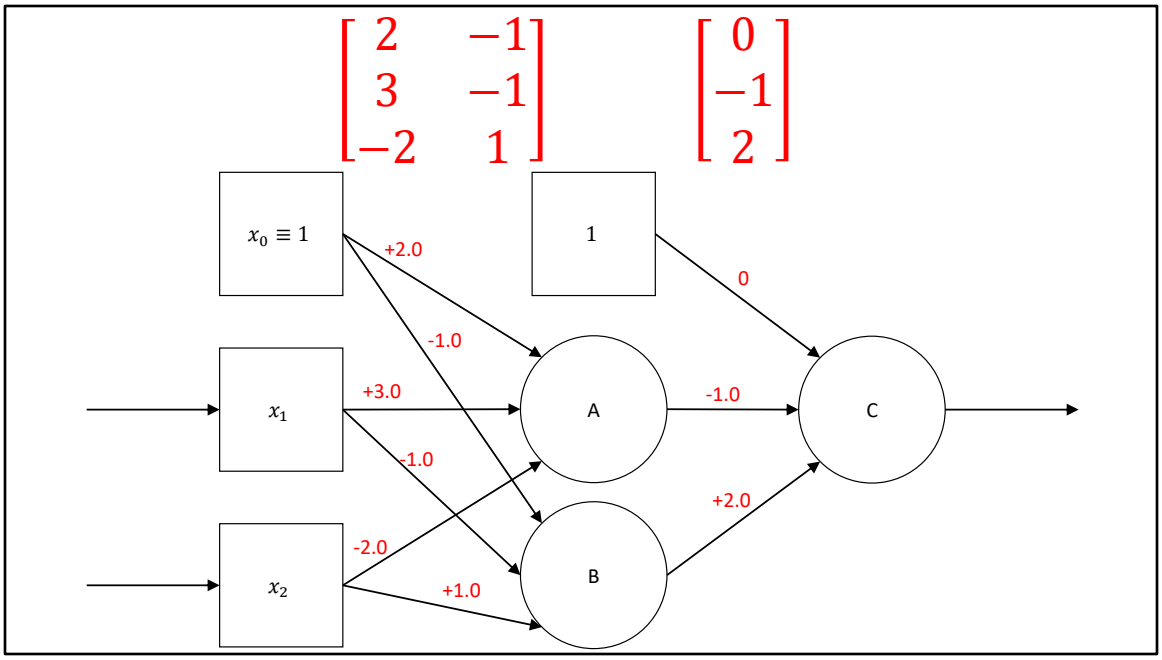



# Weights

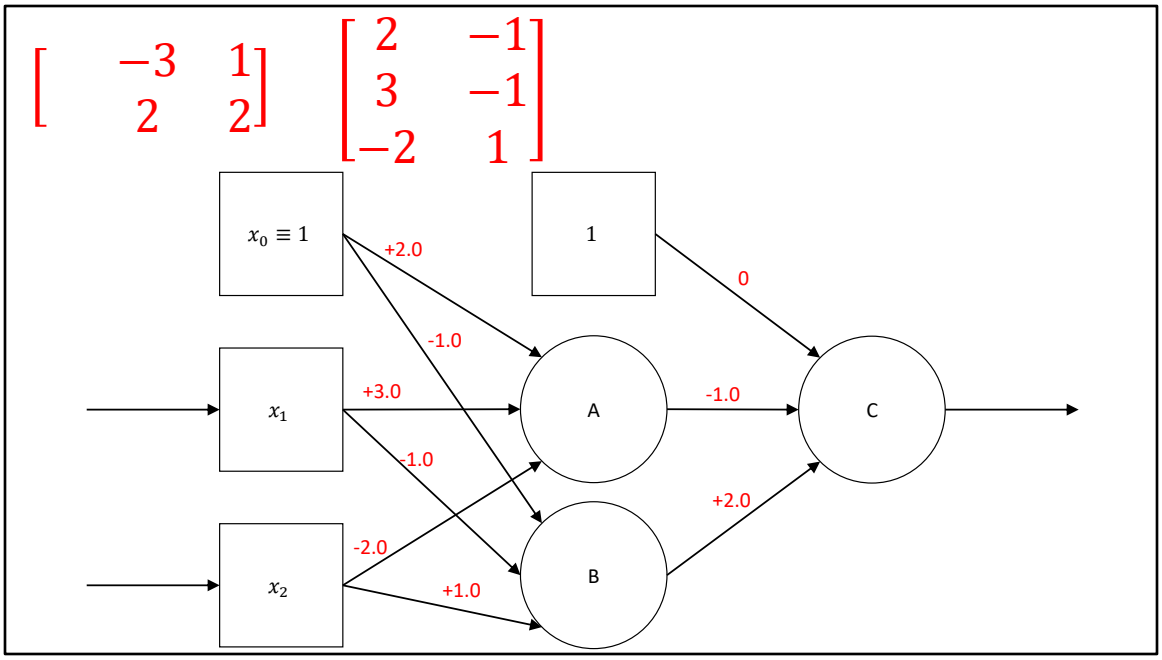
- Hyperparameters (structure and activation function)
- Parameters (weights/bias for each edge)
- Convenient to place weights into matrix


$x_0 \equiv 1$
1
+2.0
0
-1.0
$x_1$
+3.0
A
-1.0
C
-1.0
+2.0
-2.0
B
$x_2$
+1.0

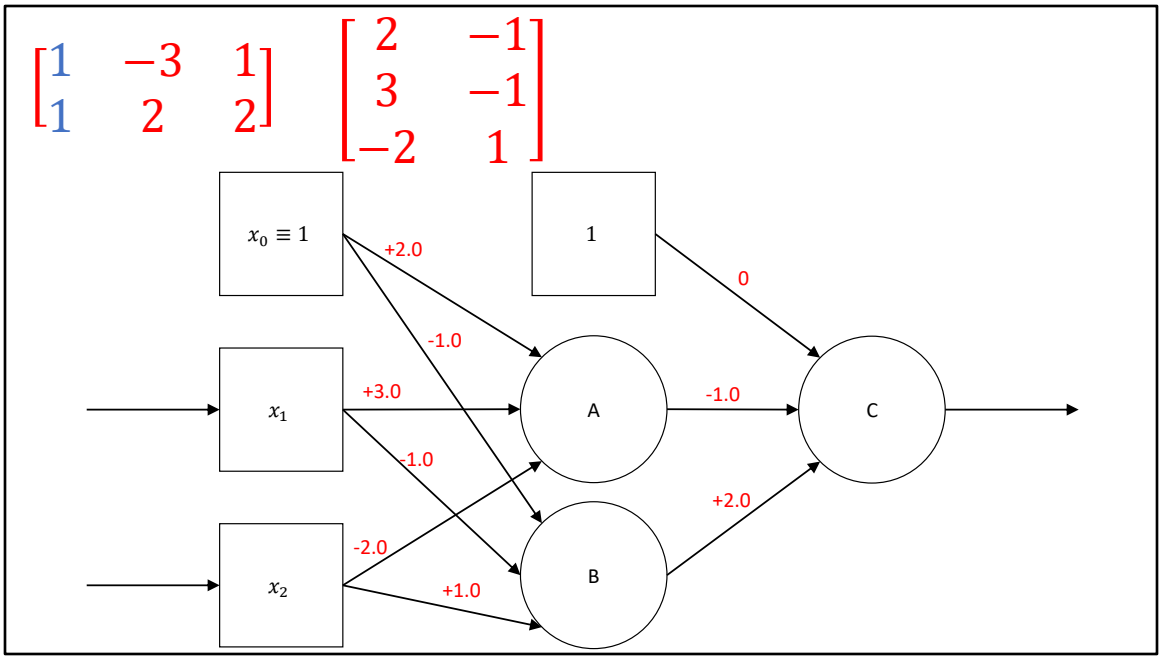

$\begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix}$
$x_0 \equiv 1$
1
+2.0
0
-1.0
$x_1$
+3.0
A
-1.0
C
-1.0
+2.0
-2.0
$x_2$
B
+1.0


$\begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix}$
$\begin{bmatrix} 0 \\ -1 \\ 2 \end{bmatrix}$
$x_0 \equiv 1$
1
+2.0
0
-1.0
$x_1$
+3.0
A
-1.0
C
-1.0
+2.0
-2.0
$x_2$
B
+1.0

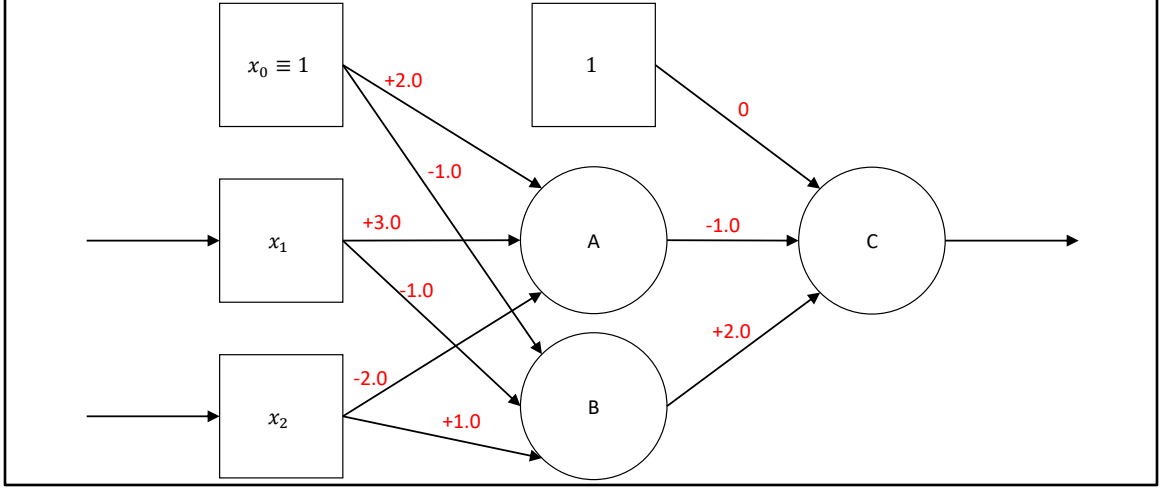
$$\begin{bmatrix} -3 & 1 \\ 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix}$$



$$\begin{bmatrix} 1 & -3 & 1 \\ 1 & 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix}$$



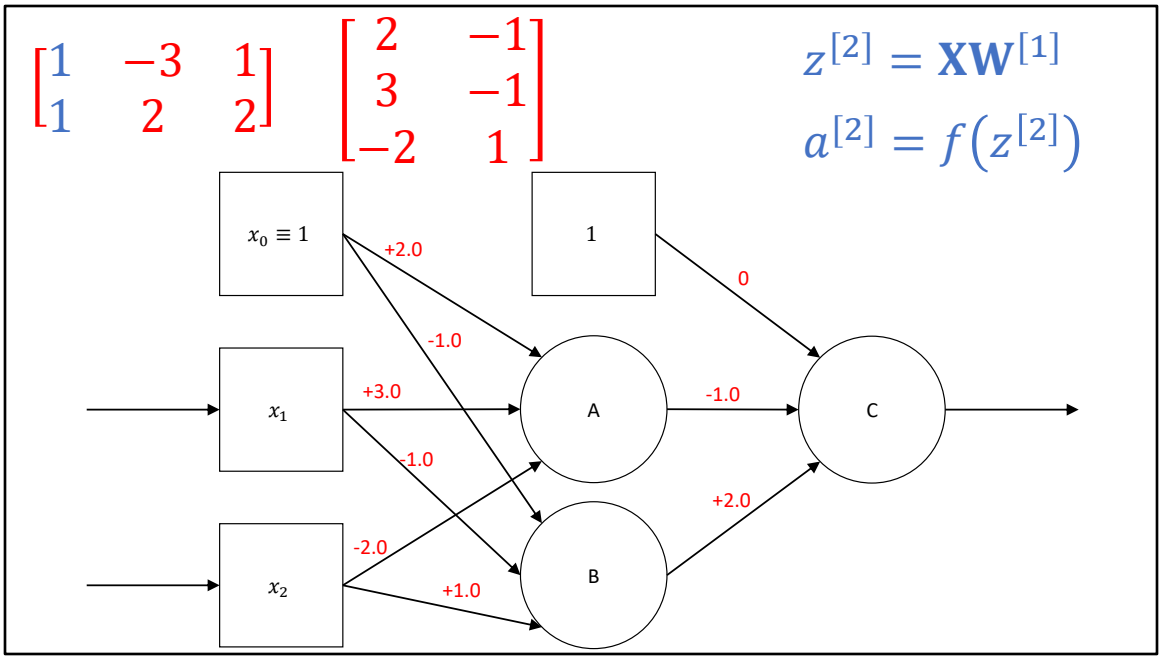
$$\begin{bmatrix} 1 & -3 & 1 \\ 1 & 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix} \quad z^{[2]} = \mathbf{X}\mathbf{W}^{[1]}$$



$$\begin{bmatrix} 1 & -3 & 1 \\ 1 & 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix}$$

$$z^{[2]} = \mathbf{X}\mathbf{W}^{[1]}$$

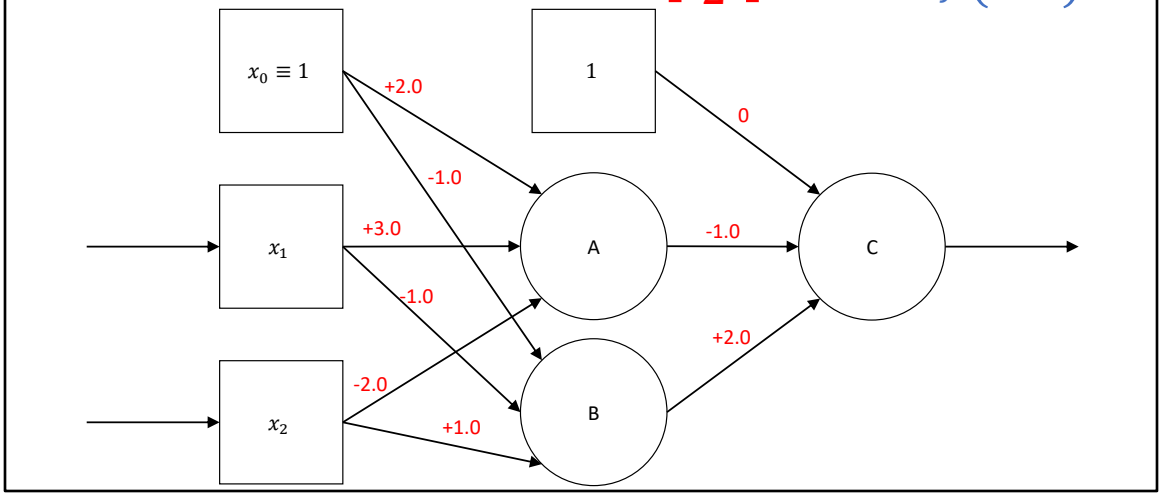
$$a^{[2]} = f\left(z^{[2]}\right)$$



$$\begin{bmatrix} 1 & -3 & 1 \\ 1 & 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 \\ -1 \\ 2 \end{bmatrix}$$

$$z^{[2]} = \mathbf{X}\mathbf{W}^{[1]}$$

$$a^{[2]} = f\left(z^{[2]}\right)$$

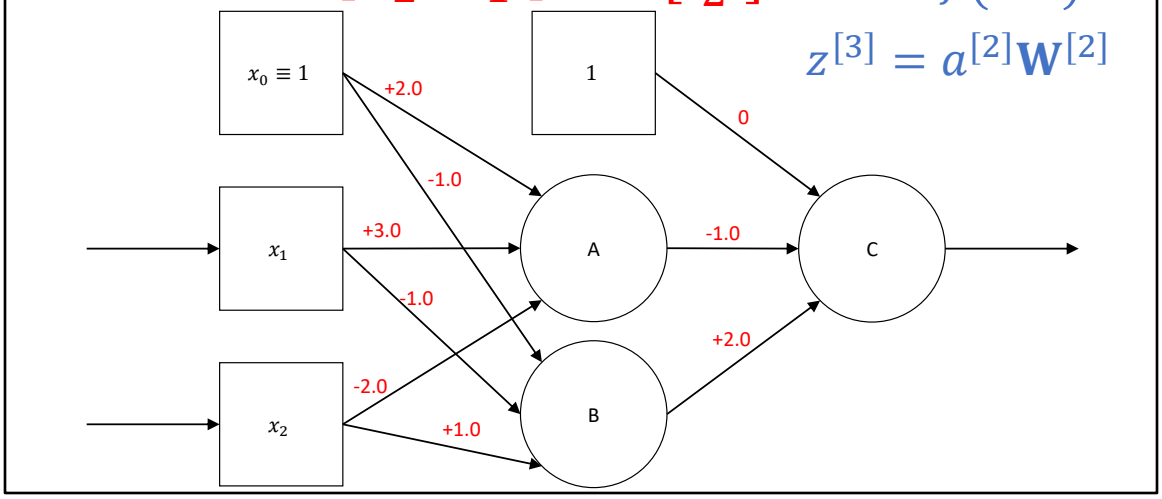


$$\begin{bmatrix} 1 & -3 & 1 \\ 1 & 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 \\ -1 \\ 2 \end{bmatrix}$$

$$z^{[2]} = \mathbf{X}\mathbf{W}^{[1]}$$

$$a^{[2]} = f\left(z^{[2]}\right)$$

$$z^{[3]} = a^{[2]}\mathbf{W}^{[2]}$$



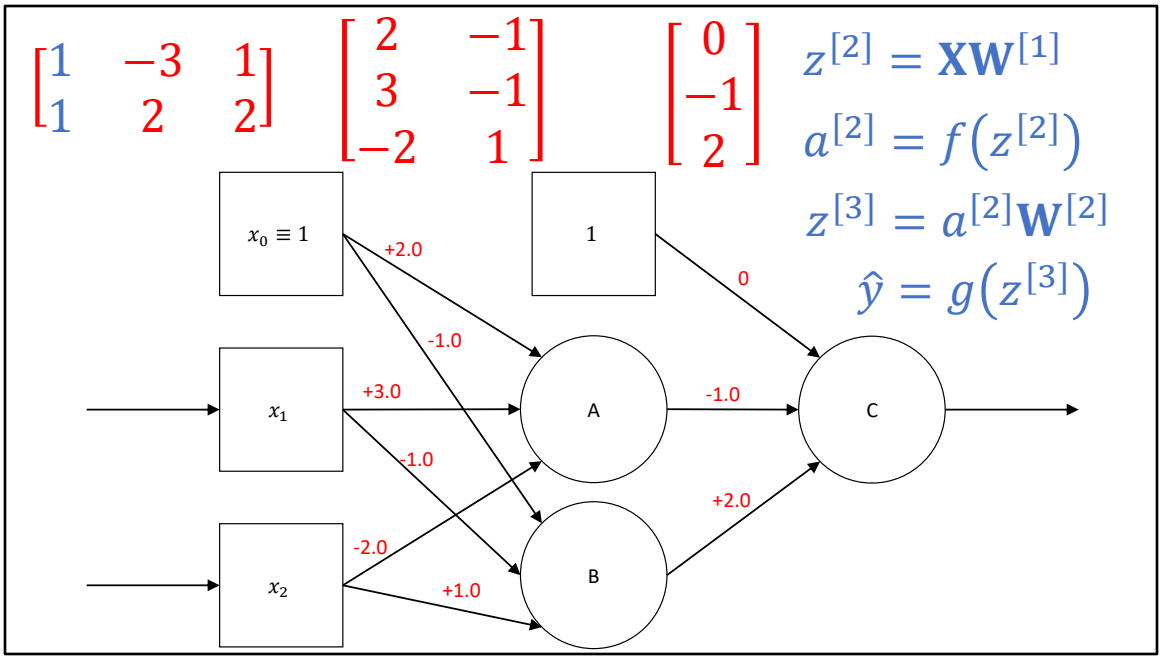
$$\begin{bmatrix} 1 & -3 & 1 \\ 1 & 2 & 2 \end{bmatrix} \quad \begin{bmatrix} 2 & -1 \\ 3 & -1 \\ -2 & 1 \end{bmatrix} \quad \begin{bmatrix} 0 \\ -1 \\ 2 \end{bmatrix}$$

$$z^{[2]} = \mathbf{X}\mathbf{W}^{[1]}$$

$$a^{[2]} = f\left(z^{[2]}\right)$$

$$z^{[3]} = a^{[2]}\mathbf{W}^{[2]}$$

$$\hat{y} = g\left(z^{[3]}\right)$$



# Learning the weights

Gradient descent and backpropagation

## Cost Function

- Compare $y$ and $\hat{y}$
- How close are we?

## Squared Loss (L2 Loss)

- Loss function for **regression**
- Measures difference between predicted and true **values**
- Formula: $L(y, \hat{y}) = (y - \hat{y})^2$
- Continuous, differentiable
- Penalizes incorrect high-confidence predictions

- **Goal: Minimize log loss for better model performance**

Log loss, also known as cross-entropy loss, is a common loss function used in binary classification problems. It measures the difference between the predicted probabilities and the true labels. The goal is to minimize this difference, leading to better model performance.

For a single data point, the log loss formula is:

L(y, p) = -[y * log(p) + (1 - y) * log(1 - p)]

where:
- y is the true label (0 or 1)
- p is the predicted probability of the positive class (1)

The log loss is calculated for each data point, and the average log loss is computed across all data points in the dataset.

The key properties of log loss are:
1. It is a continuous and differentiable function, which makes it suitable for gradient-based optimization algorithms like backpropagation.

2. It heavily penalizes incorrect predictions with high confidence. For example, if the true label is 1 and the predicted probability is close to 0, the log loss will be large.

In summary, log loss is a useful loss function for binary classification problems, as it measures the difference between predicted probabilities and true labels, and encourages the model to make accurate predictions with high confidence.

## Log Loss (Cross-Entropy Loss)

- Loss function for binary **classification**
- Measures difference between predicted **probabilities** and true **labels**
- Formula: $L(y, p) = -[y \log(p) + (1 - y) \log(1 - p)]$
- Continuous, differentiable
- Penalizes incorrect high-confidence predictions

- **Goal: Minimize log loss for better model performance**

Log loss, also known as cross-entropy loss, is a common loss function used in binary classification problems. It measures the difference between the predicted probabilities and the true labels. The goal is to minimize this difference, leading to better model performance.

For a single data point, the log loss formula is:

L(y, p) = -[y * log(p) + (1 - y) * log(1 - p)]

where:
- y is the true label (0 or 1)
- p is the predicted probability of the positive class (1)

The log loss is calculated for each data point, and the average log loss is computed across all data points in the dataset.

The key properties of log loss are:
1. It is a continuous and differentiable function, which makes it suitable for gradient-based optimization algorithms like backpropagation.

2. It heavily penalizes incorrect predictions with high confidence. For example, if the true label is 1 and the predicted probability is close to 0, the log loss will be large.

In summary, log loss is a useful loss function for binary classification problems, as it measures the difference between predicted probabilities and true labels, and encourages the model to make accurate predictions with high confidence.

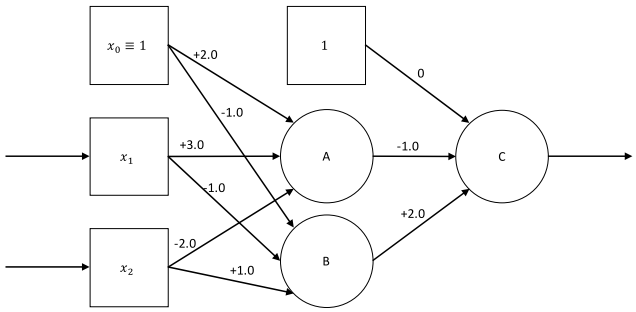
# Cost Function

- Compare $y$ and $\hat{y}$
- How close are we?
- Regression: squared error
- Classification: cross-entropy (log) loss
- Minimize cost function

$$J = \frac{1}{2N}\sum_{\text{TD}}(y - \hat{y})^2 \qquad J = \frac{1}{N}\sum_{\text{TD}}[-y\log p - (1-y)(1-\log p)\,]$$

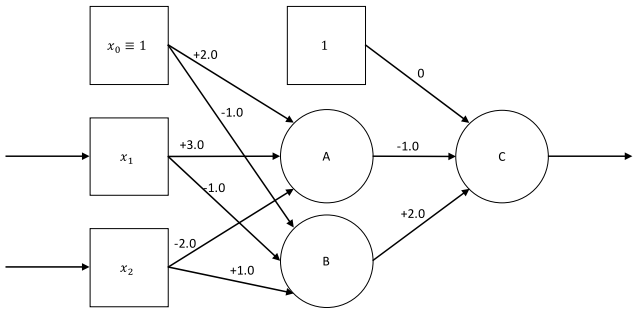
# Ideas

- Iterate over all weights?!



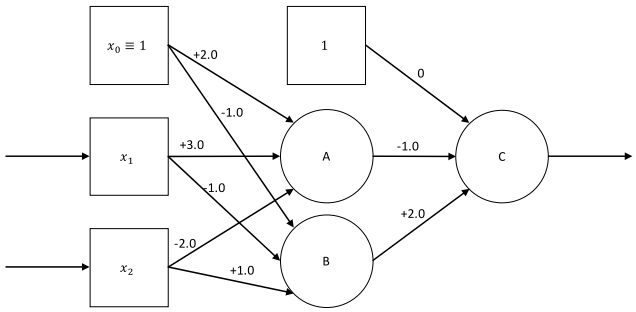
# Ideas

- Iterate over all weights?!
- Iteratively perturb weights?



# Ideas

- Iterate over all weights?!
- Iteratively perturb weights?
- Use the derivative!



## Big Idea

- Analogy: Weights of NN work as a **"team"**
- If there is **loss**, **every** weight had **some responsibility**
- Estimate responsibility due to a particular weight
- Make a **small adjustment** to each weight
- Use **derivatives** of loss (error) w.r.t. weights
  - Calc II: partial derivative $\partial a / \partial w_j$
  - Collect these in a **gradient** (same shape as weight matrix)
- Compute error derivatives for all hidden/output units simultaneously

## Go Downhill

- **Go "downhill"** (opposite gradient)
- But in $n$ dimensions (in weight space!)
- Calculus: can treat every function as a linear function
  - Only works close to the point!
  - Limitation: make **small** adjustments!
- **Gradient:** matrix of partial derivatives
  - Estimates every adjustment to correct the network
- What if hard to compute? Just test nearby points!
  - Numerical gradient estimation

## Backprop: Method Overview

- Select an **architecture**
- **Randomly** init weights
  - Unlike logistic regression, we don't start at 0
- While error too large:
  - Feedforward training set $X$ to find network output $\hat{y}$
  - Calculate and backpropagate error (chain rule)
  - Chain rule: error depends on what is down the line
  - Adjust weights
- Evaluate performance using test set

$$z^{(2)} = \mathbf{X}\mathbf{W}^{(1)}$$

$$a^{(2)} = f\left(z^{(2)}\right)$$

$$z^{(3)} = a^{(2)}\mathbf{W}^{(2)}$$

$$\hat{y} = f\left(z^{(3)}\right)$$

$$J = \sum_{\text{TD}} \frac{1}{2}\left(y - f\left(f\left(\mathbf{X}\mathbf{W}^{(1)}\right)\mathbf{W}^{(2)}\right)\right)^2$$

$$J = \sum_{\text{TD}} \frac{1}{2}\left(y - f\left(f\left(\mathbf{X}\mathbf{W}^{(1)}\right)\mathbf{W}^{(2)}\right)\right)^2$$

$$J = \sum_{\text{TD}} \frac{1}{2}\left(y - f\left(f\left(\mathbf{X}\mathbf{W}^{(1)}\right)\mathbf{W}^{(2)}\right)\right)^2$$

$$\frac{dJ}{d\mathbf{W}} = ?$$

$$J = \sum_{\text{TD}} \frac{1}{2}\left(y - f\left(f\left(\mathbf{X}\mathbf{W}^{(1)}\right)\mathbf{W}^{(2)}\right)\right)^2$$

$$\frac{\partial J}{\partial \mathbf{W}} = ?$$

$$J = \sum_{\text{TD}} \frac{1}{2}\left(y - f\left(f\left(\mathbf{X}\mathbf{W}^{(1)}\right)\mathbf{W}^{(2)}\right)\right)^2$$

$$\frac{\partial J}{\partial \mathbf{W}} > 0$$

uphill

$$\frac{\partial J}{\partial \mathbf{W}} < 0$$

downhill

# Gradient Descent

- Go "downhill"
- Iterative improvement
  - **Epoch** = one pass through training set
- Stop when costs stop improving
- Assumption: convex
  - **Stochastic gradient descent (SGD)** adjust on every instances
  - Batch gradient descent (BGD) adjust once per epoch
  - **Minibatch gradient descent (MGD)** adjust after $b$ instances

## Backpropagation (high level)

- Small random weights
- Propagate X to eval
- Compute cost J
- Backpropagate the error to each weight
- Adjust weights accordingly
- Each example has cost and gradient
- Relies on highly optimized linear algebraic library (BLAS, GPU)
- CS people: we do not model neurons as objects! We use matrix ops!

# Idea

- Compute dJ/dW (e.g., NumPy, BLAS, GPU)
- Find downhill in m-dim optimization space
- Add/subtract scalar times gradient to all weights

# Modifications

(If Time)

## Modifications

- What if >1 hidden layer?
- What if >1 class?
- When update weights?
- How avoid overfit?
- Can we speed up search?

## More than 1 Hidden Layer

- Just apply same strategy
- Update weights for inputs to first hidden layer
- Propagate error from output through 2nd hidden layer to 1st
- "Backpropagation"

## More than 1 Class

- Example: iris
- Could learn 1 network per class
- Better: 1 network, 1 output unit per class
- What is the error?
- Error for each instance: **sum** of squared errors over outputs

## Batch vs. Online Updating

- Batch Learning: Iterate over Training Data, Update Once
- Online Learning: Update after Each Instance
- With SGD overall error may temporarily go up
- **Stochastic** = regard instance $\mathbf{x}$ as **random variable** sampled i.i.d.
- Minibatch: goldilocks tradeoff between SGD and BGD

## Standardization

- Input: normalize: $\mu = 0$ and $\sigma = 1$
- Weights: init to **small Gaussian random values** with $\mu = 0$
- Why? So-called **vanishing gradient** problem
- Applies to both batch and online

## When Do We Stop?

- "Small enough"
- Overfitting (esp. for huge network)
- **Early Stopping**
  - Holdout set
  - Stop when error on validation set goes up
- **Weight Decay**
  - Same idea as **regularization constant** $C$ in logistic regression
  - Penalty: Squared sum of all nonbias weights
  - Penalizes large weights that don't "pull their weight" in reducing overall error!

## Gradient descent is slow

- **Momentum:** add small proportion of update from previous iteration
- Second derivative of error function
- Still slow

# What did we learn?

Recap

## Method at a Glance

- Select an architecture
- Randomly init weights
- While error too large:
  - Feedforward training set $X$ to find network output $\hat{y}$
  - Calculate and backpropagate error
  - Chain rule: error depends on what is down the line
  - Adjust weights
- Evaluate performance using test set

# References

- https://www.youtube.com/watch?v=bxe2T-V8XRs (and following)